SECURITY OPERATIONS LAB

WAZUH LAB SETUP: MANAGER DEPLOYMENT & AGENT ONBOARDING

PREPARED BY:

Rohan Teja

July 2026

INTRODUCTION

This report documents the base setup behind the rest of the Wazuh lab work: standing up the Wazuh manager, onboarding a Windows 10 endpoint as an agent, and adding Sysmon so the endpoint produces the kind of detailed process and network telemetry the later detection exercises (RDP brute force, Mimikatz, and so on) actually depend on.

It covers the manager deployment, the agent install and the connection issue that came with it, verification in the dashboard, and the Sysmon install using the SwiftOnSecurity configuration rather than a hand-rolled one.

Lab Environment

Machine	Role	Operating System	Purpose
rohan-VMware-Virtual-Platform	Wazuh Manager	Ubuntu 64-bit (Docker, single-node)	Runs the wazuh-indexer, wazuh-manager and wazuh-dashboard containers
DESKTOP-TDOGRJA	Agent / Endpoint	Windows 10 Pro 10.0.19045.3803	Wazuh agent plus Sysmon (SwiftOnSecurity config)

1. DEPLOYING THE WAZUH MANAGER

The manager runs as a single-node Docker deployment on Ubuntu, using the official wazuh-docker single-node compose file. Bringing the stack up starts three containers in sequence, wazuh-indexer first, then wazuh-manager and wazuh-dashboard.

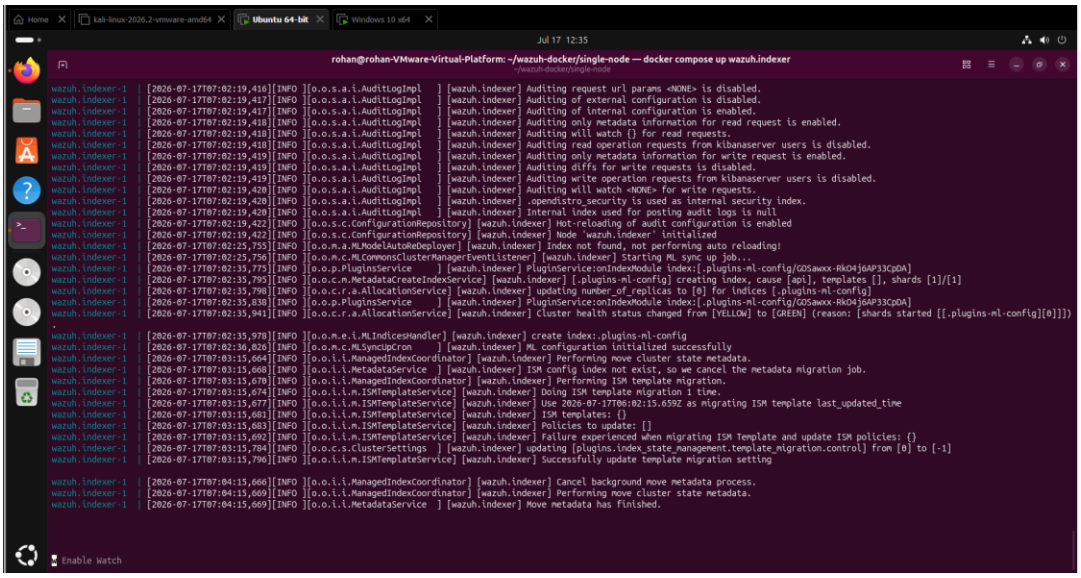

Figure 1 — wazuh-indexer container starting: audit logging, ML config, and cluster health moving from YELLOW to GREEN.

Once the indexer reports a green cluster and the dashboard container is up, the web interface becomes reachable at the manager's IP over HTTPS.

2. INITIAL DASHBOARD (NO AGENTS)

Before any agent was deployed, the dashboard's Agents Summary showed nothing registered, and the 24-hour alert counts reflected only the manager's own internal activity: 45 medium and 142 low severity events, nothing critical or high.

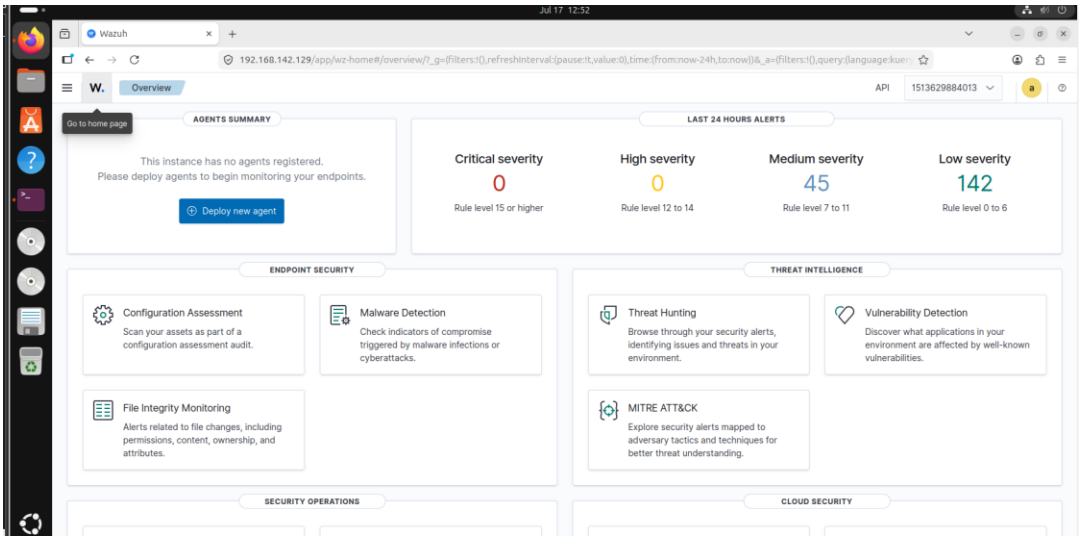

Figure 2 — Wazuh dashboard overview immediately after deployment, no agents registered yet.

3. DEPLOYING THE WINDOWS AGENT

The dashboard's “Deploy new agent” wizard generates the install command for the selected OS and group. For the Windows 10 endpoint, group Default, it produced a single PowerShell one-liner that downloads the MSI and installs it silently with the manager address and agent name baked in:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi -
OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q
WAZUH_MANAGER='192.168.142.130' WAZUH_AGENT_NAME='victim'
```

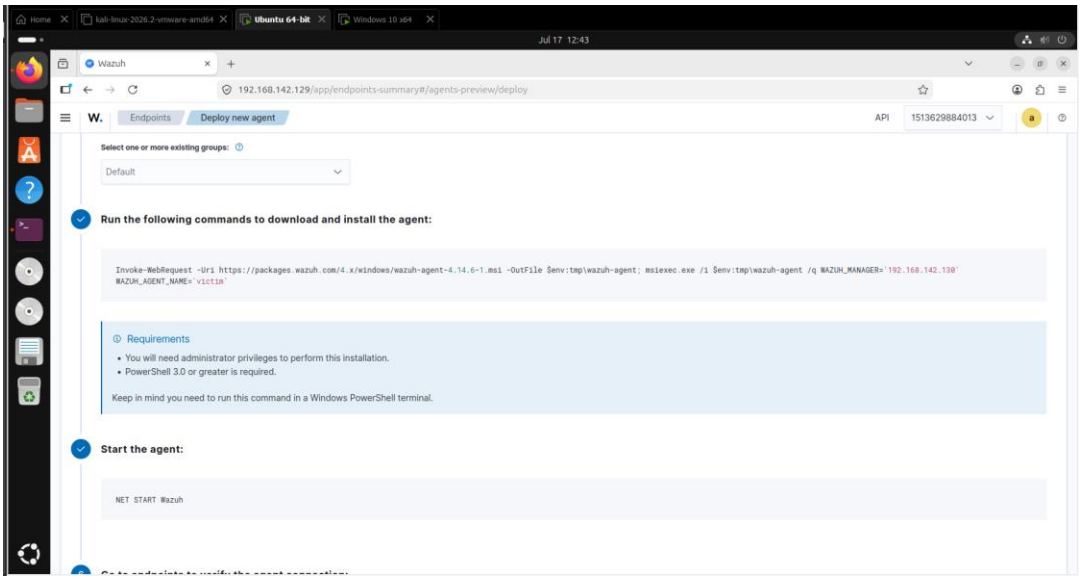

Figure 3 — Deploy new agent wizard: install command and the follow-up NET START Wazuh step.

Run as Administrator in a PowerShell terminal on the Windows endpoint, the same command downloaded and installed the agent:

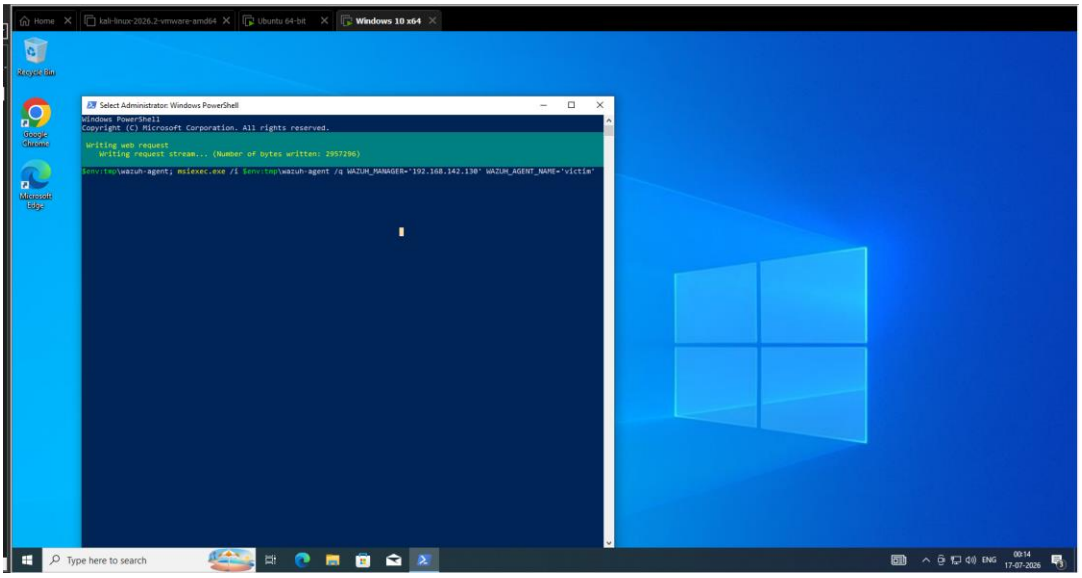

Figure 4 — Agent MSI downloading and installing on the Windows 10 endpoint.

4. TROUBLESHOOTING THE AGENT CONNECTION

Starting the service right after install didn't work. ossec.log showed the agent repeatedly failing to find a manager to talk to:

```
ERROR: (4112): Invalid server address found: '0.0.0.0'
ERROR: (1215): No client configured. Exiting.
INFO: Received exit signal. Starting exit process.
```

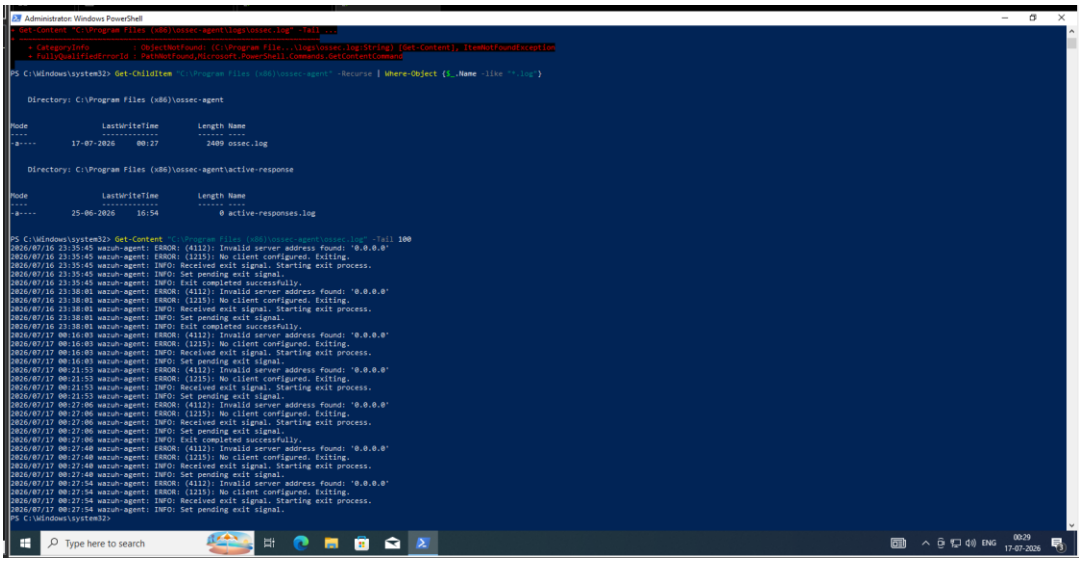

Figure 5 — ossec.log on the Windows endpoint, repeated 0.0.0.0 / no client configured errors.

The manager address hadn't actually made it into ossec.conf. The service had been started against an install that predated the WAZUH_MANAGER variable being set correctly. Re-running the documented install command with WAZUH_MANAGER='192.168.142.130' explicitly set, then starting the service again, resolved it:

```
NET START Wazuh
```

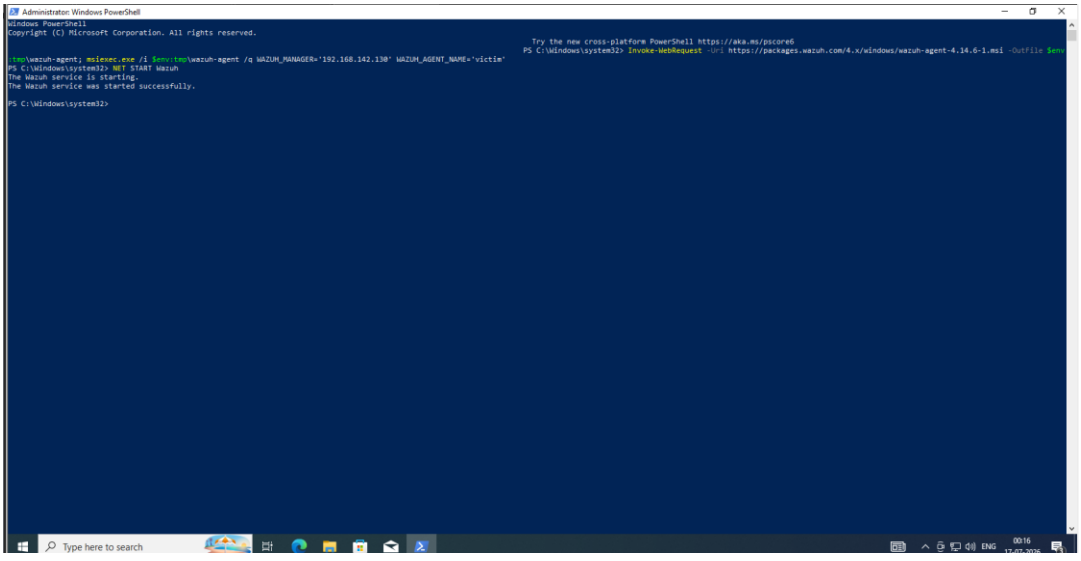

Figure 6 — Wazuh service starting successfully after the manager address was set correctly.

5. VERIFYING THE AGENT

With the service running against the correct manager address, the endpoint showed up in the dashboard as an active agent within seconds.

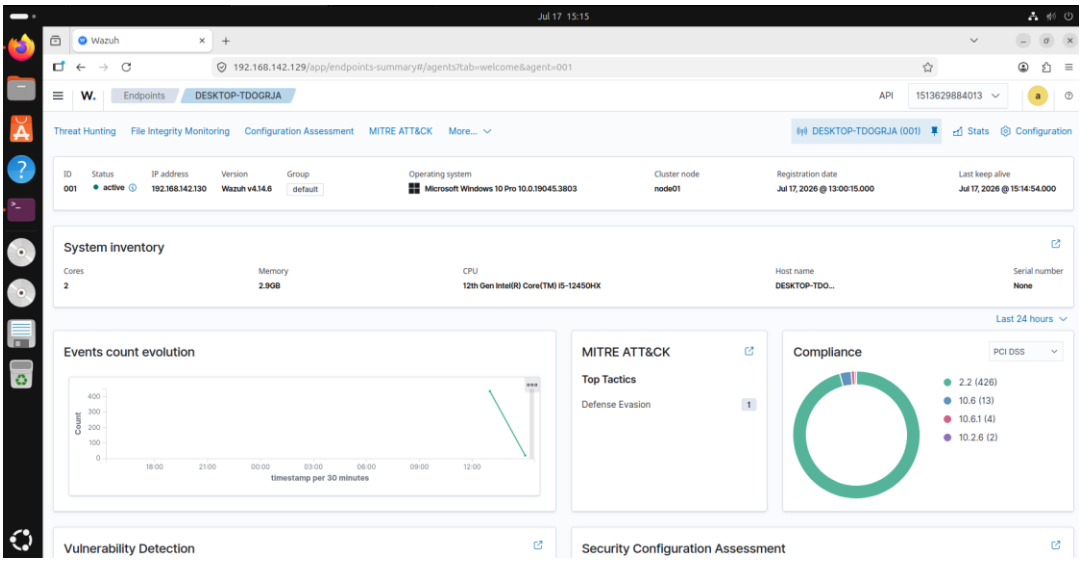

Figure 7 — DESKTOP-TDOGRJA listed as an active agent, with system inventory and event count populated.

Field	Value
Agent ID	001
Status	active
IP address	192.168.142.130
Agent version	Wazuh v4.14.6
Operating system	Microsoft Windows 10 Pro 10.0.19045.3803
Group	default
Cluster node	node01

6. ADDING SYSMON (SWIFTONSECURITY CONFIG)

The default Windows Security event log is enough for logon/logoff and account activity, but it doesn't cover process creation detail, parent/child relationships, or remote thread and memory access, which is the kind of telemetry the later detection work (Mimikatz in particular) relies on. Sysmon fills that gap, so it was installed on the endpoint after the agent was confirmed active.

Rather than writing a Sysmon config from scratch, this used the SwiftOnSecurity configuration, a well-known, widely used baseline that logs the security-relevant event types without drowning the pipeline in noise:

```
Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile
Sysmon.zip
Expand-Archive Sysmon.zip
Invoke-WebRequest -Uri https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-
config/master/sysmonconfig-export.xml -OutFile sysmonconfig-export.xml
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

With Sysmon installed and logging to Microsoft-Windows-Sysmon/Operational, the Wazuh agent config was updated to forward that channel to the manager:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

This is what later shows up as Sysmon event groups (sysmon_event1, sysmon_event8, sysmon_event_10) in Wazuh's ruleset: process creation, remote thread creation, and process memory access respectively.

7. OUTCOME

At the end of this setup: a single-node Wazuh manager running in Docker, a Windows 10 endpoint reporting in as an active agent, and Sysmon feeding process-level telemetry through the SwiftOnSecurity config. The one real hiccup was the agent starting against a stale 0.0.0.0 manager address before the install variables took effect, caught in ossec.log and fixed by reinstalling with the manager address set explicitly. This is the baseline the rest of the lab's detection work builds on top of.